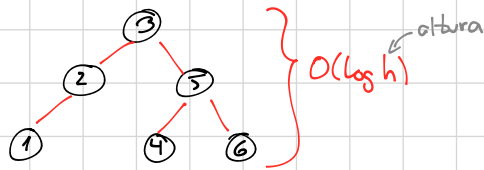


Clase 2: Árboles y heaps

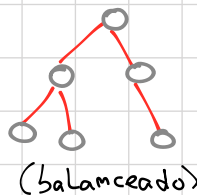
Problema 1:

- Mantener un conjunto de elementos soportando: predecesor/sucesor, inserción, eliminación, etc (min, max, search, ...)
- Problema fundamental
- Aplicaciones: Ordenamiento, colas de prioridad, diccionarios, ...

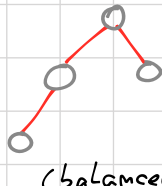
+ Solución 1: Árbol Binario de Búsqueda (BST)



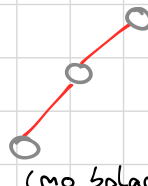
- Idealmente, mantener el BST balanceado
- Ejemplo de BST balanceado: AVL (Adelson, Velskii, Landis. 1962)
- Propiedad AVL: Un nodo está balanceado si la altura de sus subárboles izquierdo y derecho no difieren en más de 1



(balanceado)



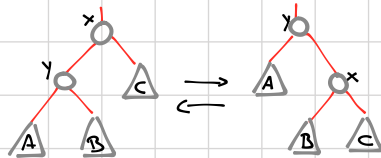
(balanceado)



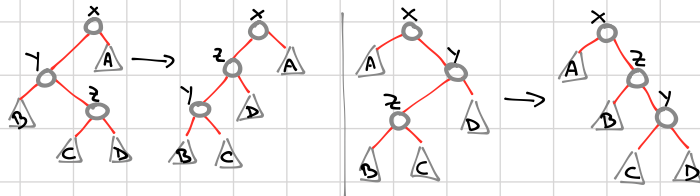
(no balanceado)

- Inserciones/Eliminaciones pueden romper el balanceo (invariante)
- Las rotaciones permiten mantener el balanceo

+ Rotación simple:



+ Rotación doble:



+ Solución 2: Red-black trees [R. Bayer 1972]

- Idea clave: Relajar la propiedad de balanceo

- Ahora tendremos árboles un poco menos balanceados

+ Inserciones/Eliminaciones más rápidos

+ Búsquedas más lentas (pero óptimas)

- Entonces ¿Para qué usaremos AVL?

+ Dominio estático o, en general, consulta intensiva

- Propiedades:

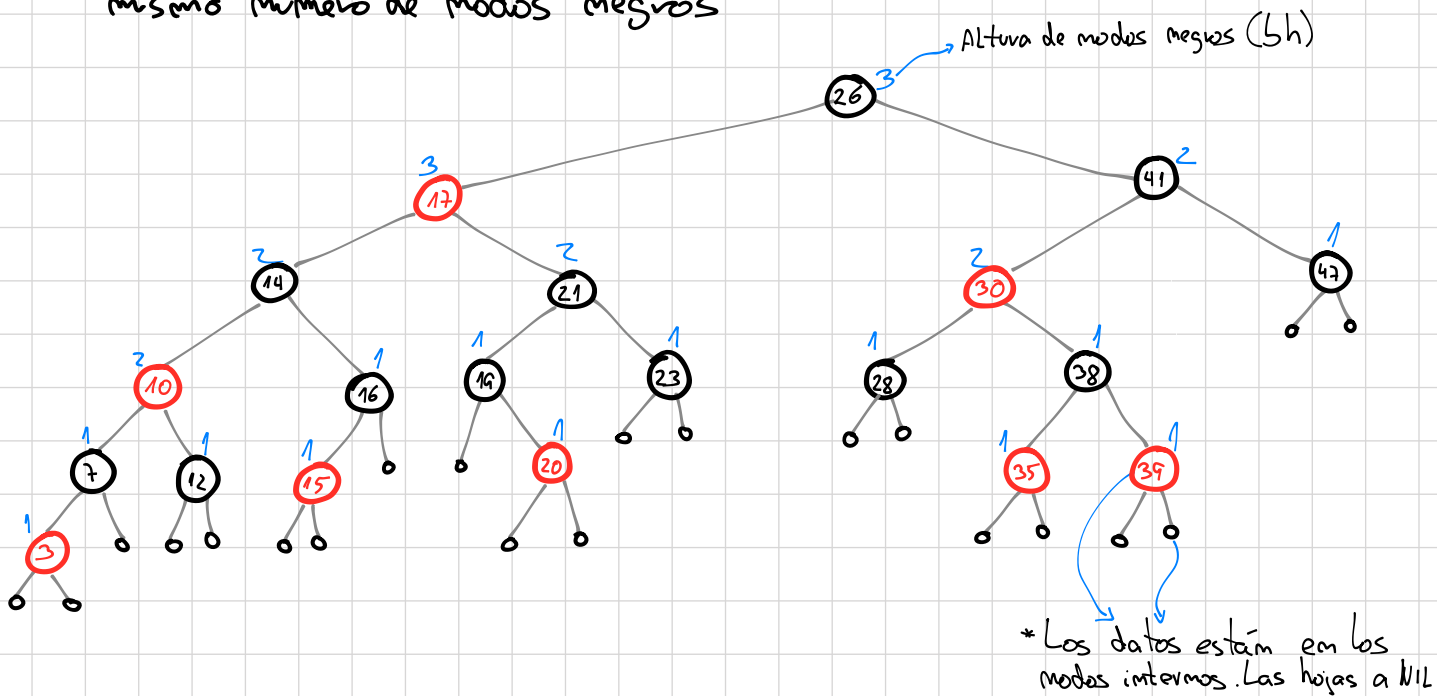
1. Cada nodo es rojo o negro (1 bit extra)

2. La raíz es negra

3. Las hojas (NIL) son negras

4. Los 2 hijos de un nodo rojo son negros

5. Para cada nodo, todo camino hasta una hoja descendiente contiene el mismo número de nodos negros



- Las propiedades garantizan altura logarítmica $O(\log n)$

Lema: Un árbol red-black con m nodos internos (claves) tiene una altura máxima de $2 \log(m+1)$

Prueba:

- + Sea $bh(v)$ el número de nodos negros, excluyendo a v , en cualquier camino hasta una hoja (propiedad 5)
- + El subárbol con raíz v tiene al menos $2^{bh(v)} - 1$ nodos internos
 - Inducción en la altura de v
 - Caso base: Altura 0 $\rightarrow$ hoja $\rightarrow 2^0 - 1 = 2^0 - 1 = 0$ nodos internos
 - Paso inductivo: v es nodo interno con 2 hijos de altura $bh(v)$ o $bh(v)-1$ según el color
 - Cada hijo de v tiene al menos $2^{bh(v)-1} - 1$ nodos internos
 - $\Rightarrow v$ tiene al menos $(2^{bh(v)-1} - 1) + (2^{bh(v)-1} - 1) + 1 = 2^{bh(v)} - 1$ nodos internos
- + Sea h la altura del árbol, por propiedad 4, al menos la mitad de los nodos de cualquier camino de la raíz a una hoja son negros.
 $\Rightarrow bh(\text{raíz}) \geq h/2 \Rightarrow m \geq 2^{h/2} - 1 \Rightarrow \log(m+1) \geq \frac{h}{2} \Rightarrow h \leq 2 \log(m+1)$

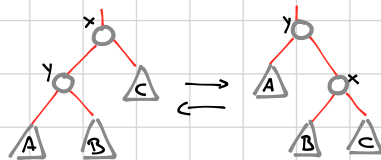
$m \geq 2^{bh(\text{raíz})} - 1$

▀

Corolario: Predecesor/Sucesor requieren tiempo $O(\log m)$

- Inserciones/Eliminaciones requieren rotaciones (similar a AVL)

+ Rotaciones



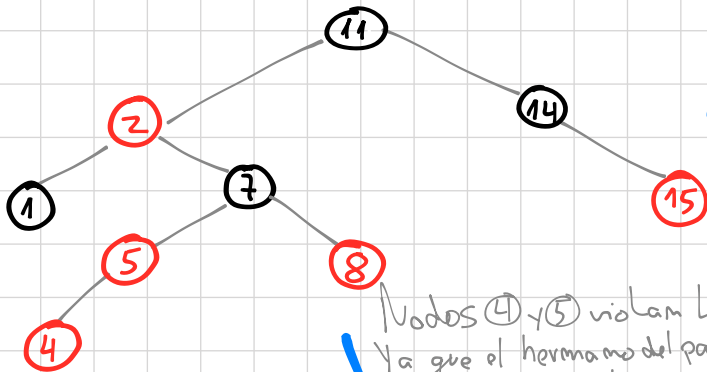
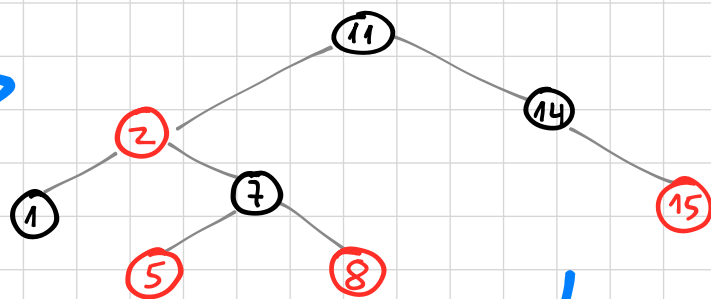
+ "Arreglar" color. Siempre insertamos primero un nodo rojo

- Insertar un nodo negro siempre viola propiedad 5
- Insertar un nodo rojo respeta prop. 5, pero puede violar prop. 4, la cual se puede verificar y reparar en $O(1)$

[Ver casos posibles en páginas 315-322 Cormen]

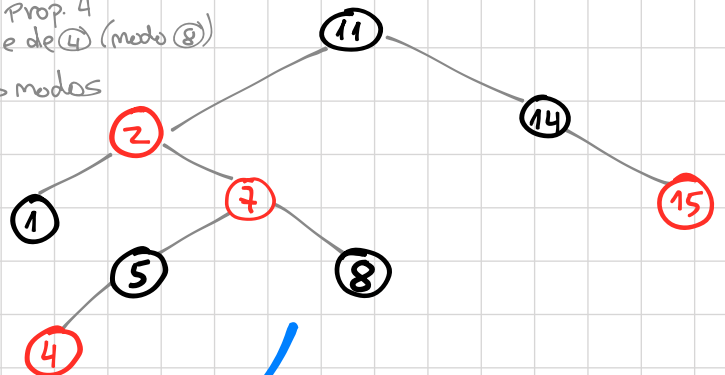
- Casos:

insert(4)



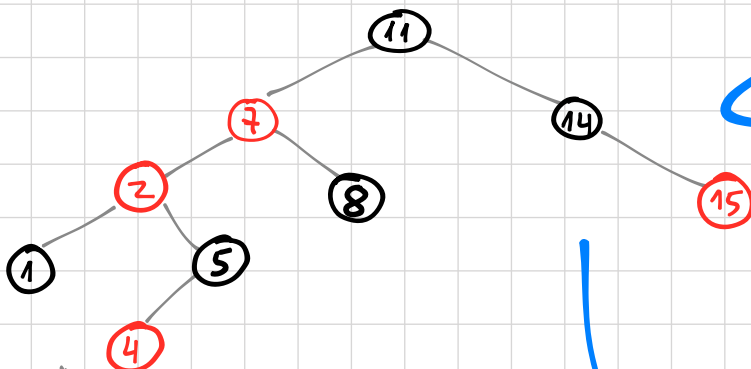
Nodos 4 y 5 violan la prop. 4
Ya que el hermano del padre de 4 (nodo 8)
es rojo, se recolorean los nodos
4, 5, 7, 8

Caso 1



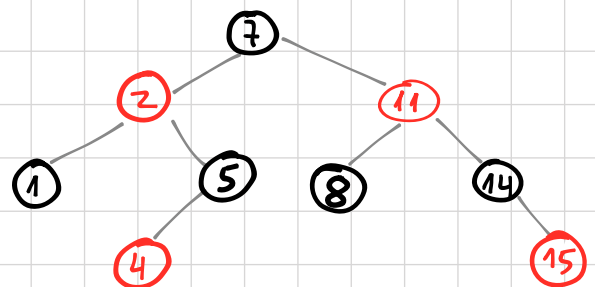
Caso 2

Nodos 2 y 7 violan prop. 4, pero
ahora el hermano del padre de 2 es negro
y 7 es hijo derecho, se realiza una
rotación izquierda



Nodos 2 y 7 violan prop. 4, pero
ahora el hermano del padre de 7 es negro
y 2 es hijo izquierdo, se realiza una
rotación derecha y se recolorea

Caso 3



- Implementaciones:

- + JDSL (Java)
- + Java collections (TreeSet, TreeMap) [Java SE]
- + LEDA (C++)
- + STL C++ (std::set, std::map)

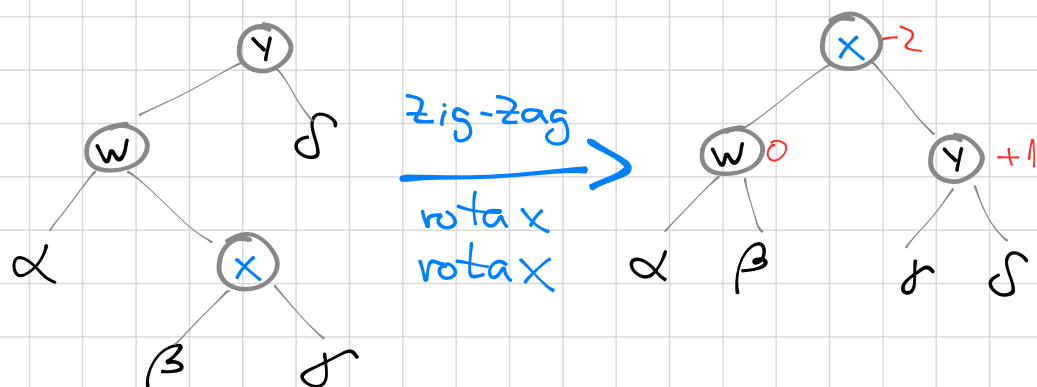
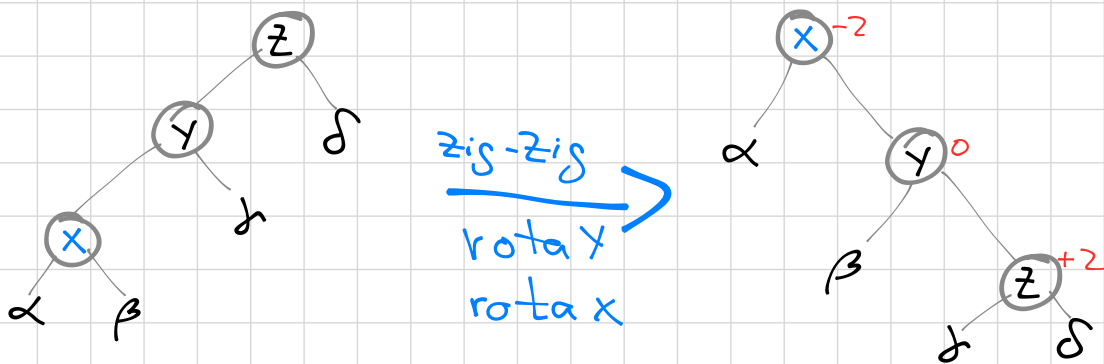
+ Solución 3 : Splay tree [Tarjan y Sleator, 1985]

- Idea: Elementos consultados recientemente pueden ser reconsultados muy rápido $\Rightarrow$ Se mueven cerca de la raíz.

- En la práctica funcionan muy bien incluso cuando la secuencia de consultas no presenta un patrón conocido
+ La estructura descubre el patrón y se auto-ajusta

- ¿Por qué funciona bien? Localidad de referencia

- Se basa en rotaciones llamadas "splay": zig-zig y zig-zag, para mover la clave consultada, x , hasta la raíz



- En el peor caso, la altura puede ser lineal para búsqueda

- Con análisis amortizado obtenemos $O(\log n)$ para secuencia de inserciones
+ Intuición: a lo más, la mitad de los nodos descienden en el árbol durante las rotaciones.

+ Implementaciones

- Sleator: Java y C.

Posibles proyectos relacionados

- Presentación más formal de splay trees (incluyendo análisis amortizado), poner en el contexto de algoritmos online y ver alternativas
- Optimalidad dinámica ¿Existe un BST tan bueno como cualquier otro para cualquier instancia?
↳ Ver apuntes "Dynamic Optimality" de Erik Demaine

Problema 2:

- Mantener un conjunto de elementos bajo las operaciones insertar, eliminar_mínimo, mínimo, etc.
- Aplicación principal: Colas de prioridad

+ Solución 1: Binary heap

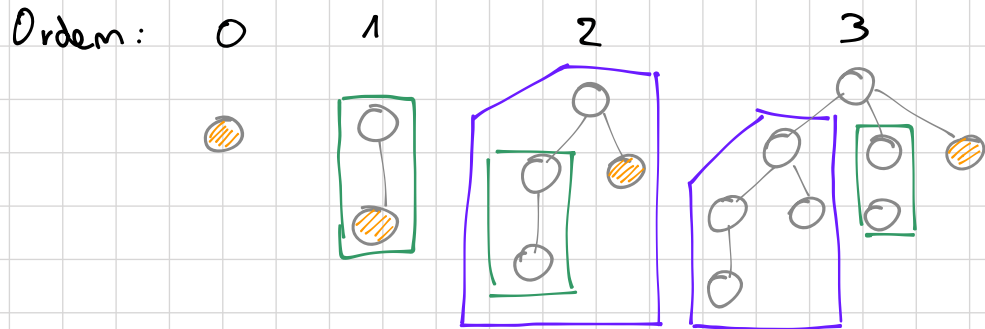
- Árbol binario completo con propiedad de heap
(para todo subárbol, la raíz tiene mayor o igual prioridad (ej. min) que sus hijos)
- Operaciones
 - + Insertar/Eliminar: $O(\log n)$
 - + Mínimo: $O(1)$
 - + Merge: $O(n)$ (Concatenar + Construcción con heapify)
- Se puede usar para ordenar
- Implementación basada en arreglos
 - + $\text{parent}(i) = \lfloor i/2 \rfloor$
 - + $\text{left child}(i) = 2i + 1$
 - + $\text{Right child}(i) = 2(i + 1)$

+ Solución 2: Binomial heap [Villlemim 1978]

- Basados en binomial trees (BT)

Definición

- Un BT de orden 0 (BT_0) es un nodo
- Un BT de orden k (BT_k) tiene una raíz cuyos hijos son BTs de orden $k-1, k-2, \dots, 1, 0$ (en ese orden)



- BT_k

+ Tiene $2^0 + 2^1 + \dots + 2^{k-1} + 1 = 2^k$ nodos

+ Altura k

+ $\binom{k}{i}$ nodos a profundidad i (de ahí su nombre)

+ Raíz de grado k

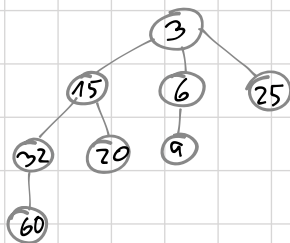
- Un binomial heap se implementa como un conjunto de BTs que cumplen las siguientes propiedades:

+ Propiedad de heap

+ Como máximo un BT de cada tipo

- Su estructura está determinada por la representación binaria del número de nodos, n

+ Ej: $n = 9|_{10} = 2^3 + 2^0|_{10} = 1001|_2$



X

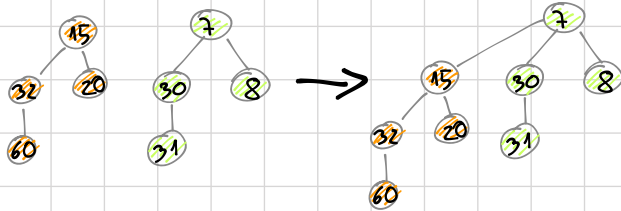
X

9

A lo más $\log n + 1$ BTs

- Operaciones

- + Mínimo: $O(\log m)$ // Comparar las raíces de los $O(\log m)$ BTs
- + Insertar: $O(1)$ amortizado // Caso especial de Merge
- + Eliminar-mínimo: $O(\log m)$ amortizado // Basado en Merge
- + Merge: $O(\log m)$ amortizado
 - Unir 2 BTs del mismo orden



- Algoritmo

- Recorrer raíces de los BT de manera simultánea
- Merge de BTs cuantos cuantos sean del mismo orden
 - + $BT_i \parallel BT_j \rightarrow BT_{i+j}$ (puede ser unido nuevamente más adelante)
- A lo más se examinan 3 BTs del mismo orden
- Análisis: Analogía con el carry en los bits de un contador
 - + Acarreos $\rightarrow$ Merge BTs

+ Solución 3: Fibonacci heaps [Fredman y Tarjan, 1974]

- Idea: Lazy binomial heaps (no forzar la forma de los árboles)
- Mejor tiempo amortizado que binomial heaps
 - + Mínimo, insertar, merge: $O(1)$ amortizado
 - + Eliminar-mínimo: $O(\log m)$ amortizado
- Implementación: LEDA

Posibles proyectos relacionados

- Fibonacci heaps en detalle
- Otros: Pairing heaps
- Evaluación experimental de distintos tipos de heaps